

CS 4782 Homework 3

Due: 3/12/26 11:59pm on Gradescope

Late submissions accepted until 3/14/26 11:59 pm

1 Byte Pair Encoding (BPE) [25 points]

Motivation

In lecture, we have been referring to the vocabulary as a collection of words. However, treating entire words as atomic units is infeasible. The English language alone has an estimated 500,000 words, not including names, variations due to plurals, conjugations, abbreviations, and other forms. Storing a unique embedding for each possible word (including typos) would require enormous memory and computational resources. Furthermore, rare words—such as technical terms or names—appear infrequently in training data, making it difficult for models to learn meaningful representations for them.

Instead of words, we use tokens which represent sub-components of words. The simplest example is just the letters of the alphabet and symbols. With those you can express any text, but your language model will be very, very slow and inefficient because you are predicting every character. Byte Pair Encoding (BPE) is an algorithm to find a good middle ground between word-level and character-level tokenization.

BPE is a data compression and tokenization algorithm that constructs new tokens by merging the most frequent adjacent character pairs. This method is widely used in modern NLP models, such as GPT and BERT, to allow for efficient representation of common words while also being able to handle unseen or rare words.

You can find an in depth explanation of the algorithm in this [HuggingFace blog](#) (highly suggested!).

BPE Algorithm

The BPE algorithm follows these steps:

1. Initialize the vocabulary, $\mathcal{V}$, to contain all the unique characters (or bytes) of the corpus.
2. Repeat the following steps until the desired vocabulary size is reached:
 - (a) Compute the frequency of all consecutive tokens/elements in the vocabulary.
 - (b) Find the most frequent adjacent pair and merge it into a new token.
 - (c) Add the new token to the vocabulary $\mathcal{V}$.

Example

Consider the following text corpus:

aaabdaaabc

Tokenized as individual characters:

$[a, a, a, b, d, a, a, a, b, a, c]$

Thus, we initialize our vocabulary as the following:

$\{a, b, c, d\}$

Now, assume we desire a vocabulary size of **6 tokens**.

1. Find the most frequent adjacent pair of tokens in the corpus: **"aa"** (appears 4 times).
2. Merge **"aa"** into a new token and update the vocabulary:

$\{a, b, c, d, aa\}$

Tokenized, the text corpus now looks like the following:

$[aa, a, b, d, aa, a, b, a, c]$

3. Again, find the most frequent adjacent pair in the updated corpus: **"ab"** or **"aaa"** (both occurring twice).
4. Merge **"aaa"** into a new token. Our final vocabulary is as follows:

$$\{a, b, c, d, aa, aaa\}$$

And our text corpus looks like the following:

$$[aaa, b, d, aaa, b, a, c]$$

Problem

1. Given the corpus **"Can you can a can as a canner can can a can?"**, apply multiple merge steps of BPE until you reach a vocabulary size of **16 tokens**. Show the resulting tokenized sequence and vocabulary after each step. Additionally, compute the ratio of the number of tokens in the tokenized sequence to the number of words in the original corpus and explain the tradeoff between the vocabulary size and tokenization "efficiency" (ie. the ratio of tokens-to-words).

Note: The space character " " should be treated as a separate token and is part of the initial vocabulary.

Initial Dictionary (of size of 12):

$$\{e, s, y, C, c, o, n, a, u, ' ', r, ?\}$$

- (I) Most frequent adjacent token pair is **"an"** with a frequency of 7. Our new vocabulary and corpus are the following respectively:

$$\{e, s, y, C, c, o, n, a, u, ' ', r, ?, an\}$$

$$[C, an, ' ', y, o, u, ' ', c, an, ' ', a, ' ', c, an, ' ', a, s, ' ', a, ' ', c, an, n, e, r, ' ', c, an, ' ', c, an, ' ', a, ' ', c, an, ?]$$

- (II) Most frequent adjacent token pairs are **"c"** and **"can"** with a frequency of 6. Our new vocabularies and corpora are the following respectively:

- i. If using **"c"**:

$$\{e, s, y, C, c, o, n, a, u, ' ', r, ?, an, ' c'\}$$

$$[C, an, ' ', y, o, u, ' c', an, ' ', a, ' c', an, ' ', a, s, ' ', a, ' c', an, n, e, r, ' c', an, ' c', an, ' ', a, ' c', an, ?]$$

- ii. If using **"can"**:

$$\{e, s, y, C, c, o, n, a, u, ' ', r, ?, an, can\}$$

$$[C, an, ' ', y, o, u, ' ', can, ' ', a, ' ', can, ' ', a, s, ' ', a, ' ', can, n, e, r, ' ', can, ' ', can, ' ', a, ' ', can, ?]$$

- (III) Most frequent adjacent token pair is **"can"** with a frequency of 6 regardless of the previously chosen token for step (II). Our new vocabulary and corpus are the following respectively:

$$\{e, s, y, C, c, o, n, a, u, ' ', r, ?, an, ' c' / can, ' can'\}$$

$$[C, an, ' ', y, o, u, ' can, ' ', a, ' can, ' ', a, s, ' ', a, ' can, n, e, r, ' can, ' can, ' ', a, ' can, ?]$$

- (IV) Most frequent adjacent token pair is **"a"** with a frequency of 4, which once again is independent of the token chosen for step (II). Our new vocabulary and corpus are the following respectively:

$$\{e, s, y, C, c, o, n, a, u, ' ', r, ?, an, ' c' / can, ' can, ' a'\}$$

$$[C, an, ' ', y, o, u, ' can, ' a, ' can, ' a, s, ' a, ' can, n, e, r, ' can, ' can, ' a, ' can, ?]$$

Thus, our corpus can be finally tokenized as the following:

$$[C, an, ' ', y, o, u, ' can, ' a, ' can, ' a, s, ' a, ' can, n, e, r, ' can, ' can, ' a, ' can, ?]$$

And the two possible vocabularies (depending on step II) are:

- (a) $\{e, s, y, C, c, o, n, a, u, ' ', r, ?, an, ' c', ' can, ' a'\}$
- (b) $\{e, s, y, C, c, o, n, a, u, ' ', r, ?, an, can, ' can, ' a'\}$

Since there were 12 words in the text and 21 tokens, this makes the ratio of tokens-to-words $\frac{21}{12}=1.75$. This means that, on average, each word in the text is represented by 1.75 tokens.

Tradeoff Between Vocabulary Size and Tokenization Efficiency: a larger vocabulary suffers from (1) and (2) but benefits from (3) and (4):

- (I) Requires more training data to ensure each token appears frequently enough for the model to learn meaningful representations.
- (II) Increases memory usage and computation for storing embeddings.
- (III) Fewer tokens per word $\rightarrow$ more efficient tokenization.
- (IV) Reduces sequence length, lowering computational cost.

A smaller vocabulary suffers from (1), (2) and (3) but benefits from (4):

- (I) More token fragmentation $\rightarrow$ increases tokens-to-words ratio.
- (II) Strains the attention mechanism due to increased context length.
- (III) Higher computational cost during training and inference.
- (IV) Handles rare/unseen words better but leads to longer sequences.

Finding the right vocabulary size balances tokenization efficiency with meeting computational and data constraints.

2. Apply the dictionary derived in (a) to the following sentence **"you could preserve a can as a canner preserves a can"**. What does this tell you about the importance of the corpus in the creation of the dictionary?

After we apply the dictionary from (a), we get either of the following tokenized corpus:

$[y, o, u, ' ', c, o, u, l, d, ' ', p, r, e, s, e, r, v, e, ' ', a, ' ', c, a, n, ' ', a, ' ', s, ' ', a, ' ', c, a, n, ' ', n, e, r, ' ', p, r, e, s, e, r, v, e, ' ', a, ' ', c, a, n, ']$

$[y, o, u, ' ', c, o, u, l, d, ' ', p, r, e, s, e, r, v, e, ' ', a, ' ', c, a, n, ' ', a, ' ', s, ' ', a, ' ', c, a, n, ' ', n, e, r, ' ', p, r, e, s, e, r, v, e, ' ', a, ' ', c, a, n, ']$

The ratio of tokens-to-words is $\frac{38}{11}$ or $\frac{39}{11} \approx 3.55$, more than twice that of part (a). This tells the following about the built corpus:

- (I) A dictionary is only as effective as the corpus it was built on.
 - (II) Poor corpus coverage results in suboptimal tokenization.
 - (III) A well-chosen corpus at train time greatly improves tokenization efficiency at test time.
3. Consider a new corpus: "1915 2020 1515 2020". If we apply BPE to create a vocabulary of size 8, we might get unexpected tokenization where numbers are split in unintuitive ways (e.g., "1915" $\rightarrow$ ["19", "15"]).

- (a) Apply BPE to this corpus showing each merge step. What patterns do you notice in how the numbers are tokenized?

We initially start with a dictionary of size 6, which means we need to perform 2 merging steps:

$$\{1, 9, 5, ' ', 2, 0\}$$

- (I) The most common adjacent character pair is **"20"** (appearing 4 times), which thus leads to the following corpus:

$$[1, 9, 1, 5, ' ', 20, 20, ' ', 1, 5, 1, 5, ' ', 20, 20]$$

- (II) The most common adjacent character pair of the new corpus is **"15"** (occurring 3 times), which leads to the following/final corpus:

$$[1, 9, 15, ' ', 20, 20, ' ', 15, 15, ' ', 20, 20]$$

We notice that the tokenization algorithm breaks down natural decimal structure of numbers non-intuitively, by, for example splitting 4-digits years in 2/3 tokens (e.g., "1915" $\rightarrow$ ["1", "9", "15"]). This results in tokenizations that split single numerical entities across multiple tokens in ways that don't preserve their decimal or semantic structure.

- (b) Explain why this might be problematic for a language model trying to understand numerical patterns or perform arithmetic reasoning.

There are many reasons as to why this could be problematic, but naming one is enough. Here is a non-exhaustive list:

- (I) Loss of numerical structure: When years like "1915" are split into ["1", "9", "15"], the model loses the decimal place-value understanding of the number. This makes it difficult to recognize that "1915" represents a specific year or a four-digit number.
 - (II) Inconsistent representations: The same number might be tokenized differently in different contexts, making it harder for the model to generalize numerical concepts. For example, "15" appears as a standalone token in some cases but might be split differently elsewhere.
 - (III) Arithmetic difficulty: Basic operations become challenging when numbers are fragmented in arbitrary ways. Adding or comparing years would require the model to first reconstruct the complete numbers from their non-intuitive token pieces.
 - (IV) Pattern recognition interference: The model may learn spurious correlations based on these arbitrary splits rather than meaningful numerical relationships. It might associate "15" with patterns that have nothing to do with its actual numerical value or position.
 - (V) Temporal understanding confusion: For dates and years specifically, this tokenization obscures chronological relationships, making it harder for the model to understand time sequences or historical contexts.
- (c) Propose and briefly explain a possible modification to the BPE algorithm that could improve how it handles numbers.
- Once again there are many correct answers, but here is one: The BPE algorithm could make use of a pre-processing step that identifies numbers and tokenize them in a consistent manner. Here are some example of consistent ways of tokenizing numbers:
- (I) The most naive way would be treating each digit as a separate token.
 - (II) Inversely, BPE could preserve numbers as atomic units by replacing them with special tokens (e.g., <NUM1915>) during BPE training, then restore them afterward.

2 Training Regimes [10 points]

Which of the following situations are examples of few shot, zero shot, transfer learning, supervised learning or self-supervised learning? Pick one that applies for each situation:

1. You want to generate dinner recipes with chat GPT.
Zero-shot learning
2. You need to add new employees' faces to a facial recognition system (you have 2-3 pictures per person).
Few-shot learning
3. You need a spam detection algorithm for email filtering using a large dataset of emails manually labeled as "spam" or "not spam."
Supervised learning
4. You are training a language model by predicting the next word in a sentence, with the surrounding words as context.
Self-supervised learning
5. You are fine-tuning a pre-trained image classification model (trained on ImageNet) to identify specific types of plants for a botany research project. To fine-tune the model, you use a large collection of labeled plant images.
Transfer learning

3 Contrastive Learning [20 points]

Consider the scenario where you are working with a Siamese Neural Network designed for a face verification system. A Siamese network applies the same model to two different inputs to compute two output representations. The network takes in pairs of face images and is tasked with determining whether they belong to the same person or not. We denote the anchor representation for some face with $\mathbf{x}_i$, a positive example with $\mathbf{x}_i^p$, and a negative example with $\mathbf{x}_i^n$, where

$$\mathbf{x}_i = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{x}_i^p = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad \mathbf{x}_i^n = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

We can measure the quality of the representations with loss functions such as the triplet loss or the SimCLR loss to train the network. For this problem, we define the triplet loss as follows:

$$\mathcal{L}_{\text{triplet}} = \max(0, \text{sim}(\mathbf{x}_i, \mathbf{x}_i^n) - \text{sim}(\mathbf{x}_i, \mathbf{x}_i^p) + m)$$

where $\text{sim}(\cdot)$ is some similarity function and m is some scalar margin. We let $\text{sim}(\cdot)$ be the cosine similarity and let the margin be $m = 1$.

We define the SimCLR loss as follows:

$$\mathcal{L}_{\text{SimCLR}} = -\log \left(\frac{\exp(\text{sim}(\mathbf{x}_i, \mathbf{x}_i^p)/\tau)}{\exp(\text{sim}(\mathbf{x}_i, \mathbf{x}_i^p)/\tau) + \exp(\text{sim}(\mathbf{x}_i, \mathbf{x}_i^n)/\tau)} \right)$$

where τ is some scalar temperature. We let $\tau = 1$ for this problem.

1. Calculate the triplet and SimCLR losses for these vectors and discuss what these loss values indicate regarding the proximity of the positive and negative examples to the anchor.

First, let's find the value of the cosine similarity between the positive and negative values and the anchor representation:

$$\text{sim}(\mathbf{x}_i, \mathbf{x}_i^p) = \cos(\theta) = \frac{\mathbf{x}_i \cdot \mathbf{x}_i^p}{\|\mathbf{x}_i\| \|\mathbf{x}_i^p\|} = \frac{(1)(2) + (2)(3)}{\sqrt{1^2 + 2^2} \sqrt{2^2 + 3^2}} = \frac{8}{\sqrt{65}} = 0.9923$$

$$\text{sim}(\mathbf{x}_i, \mathbf{x}_i^n) = \cos(\theta) = \frac{\mathbf{x}_i \cdot \mathbf{x}_i^n}{\|\mathbf{x}_i\| \|\mathbf{x}_i^n\|} = \frac{(1)(2) + (2)(4)}{\sqrt{1^2 + 2^2} \sqrt{2^2 + 4^2}} = \frac{10}{\sqrt{100}} = 1$$

Now, let's calculate the triplet loss:

$$\mathcal{L}_{\text{triplet}} = \max(0, \text{sim}(\mathbf{x}_i, \mathbf{x}_i^n) - \text{sim}(\mathbf{x}_i, \mathbf{x}_i^p) + m) = \max(0, 1 - 0.9923 + 1) = 1.0077$$

Now, the contrastive loss:

$$\mathcal{L}_{\text{SimCLR}} = -\log \left(\frac{\exp(\text{sim}(\mathbf{x}_i, \mathbf{x}_i^p)/\tau)}{\exp(\text{sim}(\mathbf{x}_i, \mathbf{x}_i^p)/\tau) + \exp(\text{sim}(\mathbf{x}_i, \mathbf{x}_i^n)/\tau)} \right)$$

$$\mathcal{L}_{\text{SimCLR}} = -\log \left(\frac{\exp(0.9923/1)}{\exp(0.9923/1) + \exp(1/1)} \right) = -\log(0.498) = 0.697$$

In terms of cosine similarity, the negative is slightly closer to the anchor than the positive instance. For the triplet loss, this means that it is within the margin and will therefore push the anchor closer to the positive and further from the negative. The SimCLR loss will be higher than the ideal case (where the positive instance is closer to the anchor than the negative instance). This loss will encourage the model to push the anchor representation closer to the positive instance and further away from the negative instance. Both losses indicate that the representations need to be improved by increasing the similarity between the anchor and positive instances, and decreasing the similarity between the anchor and negative instances.

2. Discuss the impact of increasing/decreasing the margin for the triplet loss and increasing/decreasing the temperature in the contrastive loss.

Increasing the margin:

Increasing the margin value will make the triplet loss more strict and harder to satisfy. This means that the positive instance must be even closer to the anchor than the negative instance by at least the margin

value. A larger margin encourages the model to learn more discriminative representations, pushing the positive instances closer to the anchor and the negative instances farther away. However, setting the margin too high can make the training unstable or lead to convergence issues if the constraint becomes too difficult to satisfy.

Decreasing the margin:

Decreasing the margin value will make the triplet loss easier to satisfy, as the positive instance only needs to be slightly closer to the anchor than the negative instance. A smaller margin can lead to faster convergence during training but may result in less discriminative representations. Setting the margin too low can make the loss insensitive to the relative distances between positive and negative instances, leading to suboptimal representations.

Increasing the temperature:

Increasing the temperature τ will flatten the softmax distribution, making the model less confident about the similarity scores. A higher temperature can help the model explore the representation space more broadly during training, potentially discovering better representations. However, setting the temperature too high can make the gradients too small, leading to slower convergence or instability during training.

Decreasing the temperature:

Decreasing the temperature τ will sharpen the softmax distribution, making the model more confident about the similarity scores. A lower temperature can help the model focus on the most relevant positive and negative instances, potentially leading to faster convergence. However, setting the temperature too low can cause the model to become overconfident and converge to suboptimal representations. It could also lead to vanishing gradient issues due to the softmax distribution becoming nearly one-hot.

3. What potential issues might arise with contrastive and triplet loss? What are some potential mitigation strategies? Think about the impact of selecting different negative examples on the two loss functions.
 - **Effective Pair/Triplet Mining:** Without strategic pair/triplet selection, the model may focus on easy examples that do not contribute much to learning. Advanced mining strategies are necessary to provide meaningful gradients during training.
 - **Sensitivity to Negative Selection:** The performance of both loss functions can be greatly affected by the selection of negative examples. Careful tuning is required to select negatives that are challenging but not too easy or too hard.
 - **Margin/temperature Size Impact:** An inappropriate margin size can cause poor model performance. The margin must be tuned, possibly using cross-validation, to achieve the best generalization.

These considerations are crucial when using contrastive or triplet losses in practical applications, especially in machine learning tasks involving metric learning, such as face recognition or clustering.

4 Vision Transformers (ViTs) [25 points]

1. Consider a Vision Transformer (ViT) model with an input image size of 256x256 pixels. The model divides the image into non-overlapping patches of size 16x16 pixels. Calculate the number of patches that result from dividing the image into non-overlapping 16x16 pixel patches.
 $(256/16) \cdot (256/16) = 256$ patches will be created.
2. Instead of using patches, why can't a ViT use each pixel as a token? If you choose to refer to the space or time complexity of ViTs, be specific.
The time complexity of a transformer scales quadratically with the number of tokens, so this would make training and inference very slow.
3. Why might it be useful to use a CNN to convert the image into tokens? Give at least two reasons.
Two reasons that might be mentioned are 1. The number of tokens will be reduced 2. CNNs are good at extracting local visual information, so using a CNN for tokenization can help extract local visual information from image patches, providing a better representation of the content within each patch.

4. Explain why encoding positional information in tokens is important for ViTs. Contrast ViTs with CNNs, which do not use any explicit positional encodings.
Positional encoding is crucial for ViTs because transformers lack an inherent understanding of spatial relationships and orderings in sequences. Learned positional embeddings are used by ViTs in practice because sequence length is usually fixed, unlike most natural language tasks.
5. Explain the relationship between inductive bias, training data requirements, and performance in Vision Transformers compared to CNNs.
Transformers generally require more training data than comparable CNNs as they need to learn the geometrical properties of images from scratch. The trade-off between inductive bias and the scale of training data is highlighted, where transformers, without strong assumptions about input structure, can converge to higher accuracy.